

Contents

Product Requirements Document (PRD)	1
Real-time Airfare Price Index (APIx) for India	1
1. Executive Summary	1
2. Problem Restatement (in plain terms)	1
3. Goals & Success Metrics	2
4. Scope	2
5. Stakeholders / Users	2
6. System Architecture (High-Level Flow)	3
7. Data Sources	4
8. Non-Functional Requirements	5
9. Recommended Tech Stack	5
10. Development Roadmap (Hackathon-realistic, ~5-6 week or 36-hr sprint variants)	5
11. Deliverables Checklist (mapped to “Expected Solution”)	6
12. Features to Make It Stand Out	6
13. How to Present the Idea (Pitch Structure)	7
14. Potential Judge Questions (and how to answer them)	7
15. Risks & Mitigations	8
16. Extra Domain Knowledge to Pre-Learn (Do this before building)	9
17. Team Role Suggestions (typical 6-member SIH team)	10
18. Suggested Repository Structure	10
19. Final Notes	10

Product Requirements Document (PRD)

Real-time Airfare Price Index (APIx) for India

SIH Problem Statement ID: 26056 **Organization:** Ministry of Statistics & Programme Implementation (MoSPI) — Data Informatics & Innovation Division (DIID) **Category:** Software | **Theme:** Smart Automation

1. Executive Summary

Build **APIx (Airfare Price Index)** — an automated, ethically-compliant web scraping and data pipeline system that captures real-time domestic airfares from Indian airlines and OTAs, cleans and normalizes the data, constructs a statistically sound price index, and exposes it via a dashboard and API for consumption by NSO/RBI to augment the “Transport and Communication” sub-group of the CPI.

The core value proposition: **replace sparse manual fare collection with high-frequency, high-coverage, statistically defensible automated fare tracking** that reflects what Indian travellers actually pay — including dynamic, advance-purchase-window-dependent pricing.

2. Problem Restatement (in plain terms)

- CPI currently under-represents airfare volatility because data is collected manually, infrequently, from few outlets.

- 90%+ of tickets are bought online — fares change 200–400% intraday based on demand, advance-purchase window, season, fuel surcharge.
- MoSPI needs a system that scrapes airline + OTA sites daily, cleans the data, builds an index (daily/weekly/monthly), and serves it via dashboard + API.
- Must be back-tested for 30+ days against DGCA average fare data to prove validity.

3. Goals & Success Metrics

Goal	Success Metric
Automated multi-source scraping	≥5 airlines + ≥3 OTAs scraped daily without manual intervention
Representative route coverage	≥15–20 city-pairs covering ≥60% of DGCA domestic traffic
Advance-purchase coverage	T+1, T+7, T+15, T+30, T+45 captured per route per day
Data quality	<5% missing/outlier records after cleaning
Index validity	Correlation ≥0.8 with DGCA monthly average fare (backtest)
System uptime	≥95% successful scrape jobs/day
API/dashboard usability	NSO/RBI analyst can query index in <3 clicks / 1 API call

4. Scope

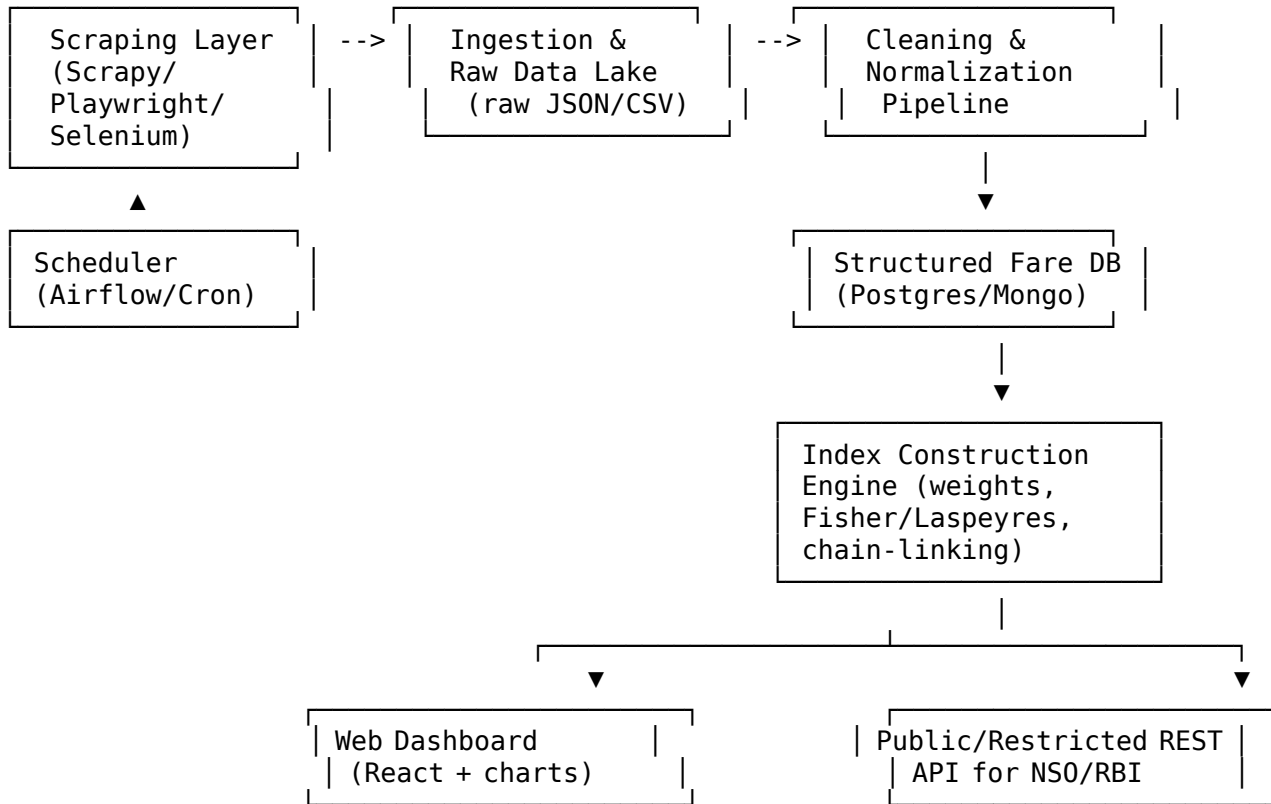
In scope (hackathon prototype): - Scraper for 5 airline sites + 2–3 OTAs (start with 2 airlines + 1 OTA fully working, rest as extensible modules) - Data cleaning & normalization pipeline - SQL/NoSQL fare database with proper schema - Index construction module (weighted, PSD/Fisher-type index) - Interactive dashboard (trends, heatmaps, elasticity curves) - REST API for index consumption - 30-day backtest vs DGCA data - Documentation + automated tests

Out of scope (mention as future roadmap, don't over-promise): - International routes - Full legal/commercial licensing agreements with OTAs - Production-grade distributed scraping infra (mention as scalable design, not fully built) - Real seat-level revenue management data (airlines won't expose this — you infer from displayed prices only)

5. Stakeholders / Users

- **Primary:** NSO/MoSPI statisticians (data producers), RBI monetary policy team (data consumers)
- **Secondary:** DGCA (route/traffic reference data owner), academic researchers, airline/OTA operators (indirectly affected)
- **Judges' lens:** Innovation + technical rigor + statistical soundness + ethical/legal compliance + real-world deployability

6. System Architecture (High-Level Flow)



Module breakdown

1. Scraping Engine

- Playwright/Selenium for JS-rendered airline booking widgets; Scrapy for OTA listing pages where static HTML suffices.
- Rotating headless-browser sessions, randomized user-agents, request throttling (rate-limiting per domain), respecting robots.txt, honoring documented API/partner terms where available.
- CAPTCHA handling: prefer **not bypassing** CAPTCHAs aggressively (legal/ethical risk) — instead design scrape frequency and patterns to avoid triggering them, use official partner APIs where offered (e.g., some OTAs have affiliate/data APIs), and build a fallback manual-refresh queue for blocked sessions.
- Proxy/IP rotation via a pool (with strict rate-limits) — but explicitly document this as a **configurable, compliance-first module**, not a mechanism to defeat legitimate anti-abuse protections.
- Scheduler: Airflow or simple cron+Celery for daily/multiple-times-a-day runs per route per advance-purchase window.

2. Data Cleaning & Normalization

- Parse raw HTML/JSON → structured record: origin, destination, carrier, flight_no, departure_date, scrape_date, advance_purchase_days, fare_class, base_fare, taxes, UDF, convenience_fee, total_fare, seat_availability_flag.
- Outlier detection (IQR / z-score per route-window), handling of sold-out/cancelled flights (flag, exclude from index, retain for availability analytics), de-duplication of repeated scrapes.

- Currency/decimal normalization, missing value imputation (carry-forward within a day, or route-level median).
 - 3. **Fare Database**
 - Relational schema (Postgres) for structured queries + time-series optimized store (TimescaleDB extension) for fast index computation.
 - Metadata tables: routes, carriers, DGCA traffic weights, index basket configuration.
 - 4. **Index Construction Module**
 - Basket: representative city-pairs weighted by DGCA passenger-traffic share (mirrors how CPI item-baskets are weighted by consumption share).
 - Methodology: modified Laspeyres (base-period quantity weights) is what CPI traditionally uses; consider a **Fisher Ideal Index** or **chained Laspeyres** for revisions — document both, recommend chained Laspeyres for consistency with existing CPI methodology, but show Fisher as robustness check (“PSD” = likely intends “Price weighted, Sector-wise, DGCA-weighted” — clarify basket construction using DGCA passenger share as weights, aggregated using a Laspeyres/Fisher price relative formula).
 - Compute at 3 frequencies: daily (raw fare index), weekly (rolling avg), monthly (aligned to CPI release cycle).
 - Advance-purchase-window sub-indices (T+1...T+45) to show lead-time elasticity.
 - 5. **Dashboard**
 - Price trend lines (daily/weekly/monthly index)
 - Sector-wise heatmap (route x price movement)
 - Lead-time elasticity curves (fare vs. days-to-departure)
 - Carrier-wise comparison
 - Anomaly/alert panel (fare spikes tied to festivals, fuel price changes)
 - 6. **API Layer**
 - REST endpoints: /index/daily, /index/monthly, /routes/{pair}/fares, /backtest/dgca-comparison
 - Auth via API key (for NSO/RBI), rate-limited public read-only summary endpoint for transparency/open-data goals.
 - 7. **Backtesting Module**
 - Pull DGCA monthly average fare data (from esankhyiki.mospi.gov.in / DGCA reports) and correlate against your constructed index for ≥ 30 days / relevant months.
 - Report correlation coefficient, MAPE (mean absolute % error), and visualize both series overlaid.
 - 8. **Documentation & Testing**
 - README, architecture diagram, data dictionary, API docs (Swagger/OpenAPI)
 - Unit tests (pytest) for cleaning functions, index math; integration tests for scraper resilience (mock HTML fixtures so tests don’t hit live sites).
-

7. Data Sources

- Airline sites: IndiGo, Air India, Air India Express, Akasa Air, SpiceJet (direct booking engines)
 - OTAs: MakeMyTrip, Yatra, EaseMyTrip, Cleartrip, Ixigo, Goibibo
 - Reference/weighting data: DGCA passenger traffic statistics, MoSPI eSankhyiki portal (<https://esankhyiki.mospi.gov.in>) for CPI methodology and validation data
 - Suggested basket (from traffic share): DEL-BOM, DEL-BLR, BOM-BLR, DEL-CCU, BLR-HYD, MAA-DEL, DEL-PNQ, BOM-GOI, DEL-HYD, CCU-BLR, etc.
-

8. Non-Functional Requirements

- **Legal/Ethical compliance:** Respect robots.txt, ToS review per source, rate-limit to avoid server strain, no personal data collection, no CAPTCHA-defeat automation that could be construed as unauthorized access under IT Act Sec. 43/66.
 - **Scalability:** Design scraper as pluggable per-source modules so adding a new airline/OTA is config-driven, not a rewrite.
 - **Resilience:** Retry logic, alerting on scrape failures, schema-change detection (site layout changes break scrapers — build a lightweight selector-health-check).
 - **Security:** API-key auth, encrypted credentials/proxy configs, no scraping of personal/PII data.
 - **Auditability:** Every index value traceable back to underlying raw quotes (important for a statistical agency).
-

9. Recommended Tech Stack

Layer	Tools
Scraping	Python, Scrapy, Playwright/Selenium, Scrapy-Playwright
Scheduling	Apache Airflow (or Celery+Redis for simpler prototype)
Storage	PostgreSQL + TimescaleDB, or MongoDB for raw JSON staging
Processing	Pandas, NumPy, Great Expectations (data quality checks)
Index/Stats	Python (statsmodels/NumPy), custom Laspeyres/Fisher index functions
Backend API	FastAPI (auto-generates OpenAPI docs)
Frontend Dashboard	React + Recharts/D3.js/Plotly
Testing	Pytest, mock HTML fixtures
Deployment	Docker Compose (prototype), documented path to Kubernetes for production
CI	GitHub Actions

10. Development Roadmap (Hackathon-realistic, ~5-6 week or 36-hr sprint variants)

If it's a multi-week SIH build phase:

1. **Week 1:** Finalize route basket using DGCA data; build 1 airline + 1 OTA scraper; raw data lake.
2. **Week 2:** Cleaning pipeline + DB schema; add 2 more sources.
3. **Week 3:** Index construction module + methodology validation with mentors/statisticians.
4. **Week 4:** Dashboard + API; begin daily automated runs to accumulate backtest data.
5. **Week 5:** Backtesting vs DGCA data, bug fixing, documentation, tests.
6. **Week 6:** Polish, demo prep, pitch deck, presentation rehearsal.

If it's the 36-hour hackathon grand finale:

- Hours 0–6: Architecture lock-in, scaffold repo, 1 working scraper end-to-end (proof of concept)
- Hours 6–14: Cleaning pipeline + DB + 2nd/3rd source scraper
- Hours 14–22: Index module (use **pre-seeded historical data** since 30-day live scraping isn't possible in 36 hrs — see note below) + dashboard skeleton
- Hours 22–30: API, backtest against DGCA sample data, polish dashboard
- Hours 30–36: Documentation, demo video, pitch rehearsal, buffer for bugs

Important pragmatic note: You cannot genuinely scrape live for 30 days during a hackathon. Two honest options: (a) run the live scraper for whatever real days you have (even 2–3 days) and supplement/backfill with a **synthetic-but-realistic historical dataset** generated from documented fare patterns + DGCA published averages, clearly labeled as simulated for demo purposes; or (b) scrape historical fare snapshots from cached/archived sources (e.g., Wayback Machine snapshots of OTA pages) — cite this as your backtest data source. Be transparent about this in your documentation; judges respect honesty about data limitations far more than an unconvincing claim of “30 days live data” that doesn't hold up under questioning.

11. Deliverables Checklist (mapped to “Expected Solution”)

- ☐ Working scraper (Scrapy/Selenium/Playwright) — scheduled daily extraction, ≥ 5 sources ideally, minimum 2–3 fully functional for demo
 - ☐ Cleaned, de-duplicated fare database with full metadata schema
 - ☐ Index construction module (documented methodology, route weights from DGCA data)
 - ☐ Interactive web dashboard (trends, heatmaps, elasticity curves)
 - ☐ Public/NSO-facing REST API with OpenAPI docs
 - ☐ 30-day (or clearly-labeled equivalent) backtest report vs DGCA data
 - ☐ README + architecture doc + data dictionary
 - ☐ Automated test suite (unit + integration with mocked HTML)
 - ☐ Ethics/compliance note (robots.txt adherence, rate-limiting policy, ToS review summary)
-

12. Features to Make It Stand Out

1. **Statistical rigor panel** — show your index methodology transparently (Laspeyres vs Fisher comparison, weight derivation from DGCA data) — most teams will just scrape and plot; showing you understand *index number theory* (the actual CPI methodology MoSPI uses) is a huge differentiator.
2. **Anomaly explainability** — auto-tag fare spikes with likely causes (festival calendar, fuel ATF price data, school holidays) using a rules-engine or simple ML classifier — turns raw numbers into a story.
3. **Fare-manipulation / “dark pattern” detector** — flag suspicious pricing behavior (e.g., price-cookie tracking, artificial scarcity messages like “3 seats left”) as a bonus consumer-protection angle — MoSPI/DIID would find this compelling.
4. **Lead-time elasticity calculator** — a genuinely useful, less-commonly-built module: shows optimal booking window per route, valuable beyond just CPI (a public good tool).
 - 4a. **Regional/tier-2 city coverage** — most teams will only do metro routes; adding tier-2 city pairs (e.g., patna, ranchi, guwahati routes) shows deeper problem understanding of “Transport & Communication” sub-group relevance across income groups.

5. **Confidence-interval-bound index** — instead of a single index number, publish it with a statistical confidence band (reflecting scrape completeness/data quality that day) — signals genuine data-science maturity.
 6. **Open API + transparency layer** — publish methodology + sample data publicly (with appropriate redaction) — aligns with government's Open Government Data philosophy, a strong pitch point for a MoSPI-sponsored PS.
 7. **Graceful degradation & self-healing scrapers** — auto-detect when a site's HTML structure changes (selector health check) and alert instead of silently failing/corrupting data — shows production-readiness thinking.
 8. **Modular “add a new source in 1 config file” architecture** — demonstrates scalability without over-engineering during the hackathon.
-

13. How to Present the Idea (Pitch Structure)

Recommended pitch flow (aim for ~7-8 min presentation + Q&A):

1. **Hook (30 sec):** Open with a real, dramatic example — “A DEL-BOM ticket can cost ₹3,200 today and ₹11,000 next Tuesday for the same seat. CPI doesn't see that volatility today.”
2. **Problem (1 min):** Manual collection gap, 90% of tickets sold online, RBI's inflation-targeting needs accurate, high-frequency data.
3. **Solution overview (1 min):** One architecture slide — scrape → clean → index → dashboard/API.
4. **Live demo (3 min): THIS IS THE MOST IMPORTANT PART.**
 - Show scraper running / recent scrape logs
 - Show cleaned data table
 - Show the dashboard: trend line, heatmap, elasticity curve
 - Show the backtest chart overlaying your index vs DGCA average fare — this is your credibility moment
 - Hit an API endpoint live (Postman/browser) and show JSON response
5. **Methodology deep-dive (1 min):** Briefly explain index formula and weighting — shows rigor without losing non-technical judges.
6. **Ethics & compliance (30 sec):** Explicitly state your robots.txt/rate-limiting/ToS approach — MoSPI judges will care about this a lot given it's a government-legal-sensitive scraping problem.
7. **Roadmap & impact (30 sec):** How this scales to full national deployment, potential to expand to rail/bus fares (Transport sub-group broadly), cost savings vs manual collection.
8. **Close (15 sec):** Restate the core value line + open for questions.

Presentation tips: - Lead with the live demo, not slides — judges remember working software.
- Have a fallback recorded video demo in case of live wifi/scraping failures during presentation (airline/OTA sites may rate-limit you at the worst time). - Keep one slide entirely about “what we didn't do and why” (scope honesty) — this builds credibility.

14. Potential Judge Questions (and how to answer them)

Question	Suggested Answer Direction
"Is this legal? Are you violating airline/OTA ToS?"	Explain your robots.txt compliance, rate-limiting, no-CAPTCHA-bypass policy, and that MoSPI as a statistical agency has a public-interest data-collection mandate; mention you'd pursue formal data-sharing MoUs with OTAs/DGCA in a real deployment rather than perpetual scraping.
"How do you handle site structure changes breaking your scraper?"	Selector health-checks, alerting, modular per-source scraper design, versioned parsers.
"How is your index methodology different from/consistent with the existing CPI methodology?"	Explain Laspeyres/Fisher choice, DGCA-traffic-based weights mirroring CPI's expenditure-weight logic, and how it plugs into the existing Transport & Communication sub-group without disrupting overall CPI computation.
"Only 30 days of backtest — is that statistically significant?"	Acknowledge limitation honestly; explain correlation/MAPE metric used, and describe how significance improves with longer-run deployment; distinguish between hackathon-time constraints and production plan.
"What happens when an airline blocks your scraper entirely?"	Fallback: reduce frequency, use cached/partner API where available, flag data gap in index confidence interval rather than fabricate.
"How do you separate a genuine price change from a data-quality artifact (bug, layout change, wrong currency parse)?"	Outlier detection, cross-source validation (compare same route/date across 2+ OTAs), anomaly flagging pipeline.
"Why should RBI trust a scraped index over official data?"	Frame it as <i>augmentation</i> , not replacement — feeds a leading, high-frequency indicator layered on top of/validated against official manual collection and DGCA data.
"How do you handle regional carriers or seasonal routes not in DGCA top routes?"	Explain basket-review mechanism — periodic re-weighting like CPI base-year revisions.
"What's your data storage/privacy stance?"	No PII collected — only publicly displayed fare listings, no login-walled personal data scraped.
"Can this scale to real-time (sub-daily) index updates?"	Yes — architecture supports higher-frequency scraping; discuss trade-off between frequency and risk of being blocked/rate-limited.

15. Risks & Mitigations

Risk	Mitigation
IP bans / CAPTCHA blocks	Conservative rate-limiting, session rotation within ethical bounds, fallback data sources

Risk	Mitigation
Site layout changes break scrapers	Modular selector config, automated health checks, alerting
Legal/ToS ambiguity	Document compliance approach explicitly; propose formal MoU path for production
Data quality issues skewing index	Outlier detection, cross-source cross-validation, confidence intervals
Insufficient real backtest data during hackathon	Be transparent; use archived/cached snapshots or clearly-labeled synthetic backfill
Judges perceive it as “just a scraper”	Emphasize statistical methodology, index theory, and consumer-facing insights (differentiators in Section 12)

16. Extra Domain Knowledge to Pre-Learn (Do this before building)

1. CPI & Index Number Theory Basics

- Understand Laspeyres, Paasche, and Fisher Ideal Index formulas and when each is used.
- Understand how MoSPI’s actual CPI is constructed (base year, weighting diagrams, sub-group structure — “Transport and Communication” is one of the 6 CPI sub-groups). Review the CPI methodology manual available via MoSPI/eSankhyiki.
- Understand chain-linking and base-year revision concepts.

2. DGCA Data

- Familiarize with DGCA’s monthly “Domestic Air Traffic” and “Air Fare” reports/statistics — these are your ground truth for basket weighting and backtesting.
- Understand what data DGCA publishes (traffic by route/airline, load factors) vs. what it doesn’t (real-time fares) — this is precisely the gap your project fills.

3. RBI Monetary Policy Context

- Basic understanding of flexible inflation targeting (RBI’s $4\% \pm 2\%$ CPI target) and why timely, granular CPI sub-indices matter for policy decisions.

4. Web Scraping Legal/Ethical Landscape in India

- IT Act 2000 provisions relevant to unauthorized access (Sec. 43, 66) — understand the line between public-data scraping and “unauthorized access.”
- robots.txt and ToS conventions — know how courts (including US cases like hiQ vs LinkedIn, referenced globally) have treated scraping of publicly visible data, for context in Q&A.
- Rate-limiting/ethical scraping best practices (crawl-delay, off-peak scheduling, honoring Disallow directives).

5. Technical Pre-requisites

- Practice with Playwright/Selenium against JS-heavy booking engines (many airline sites use React/Angular SPAs with dynamic fare calendars).
- Learn about proxy pool services and anti-bot detection techniques conceptually (to design around them ethically, not to build aggressive evasion tools).
- Time-series database basics (TimescaleDB) if going beyond plain Postgres.
- FastAPI + OpenAPI/Swagger for quick, well-documented API delivery.

6. Airfare Industry Structure

- Understand fare components: base fare, fuel surcharge (now often merged into base fare), UDF (User Development Fee, airport-specific), CUTE charges, GST — needed to correctly design your “base fare vs taxes vs total” schema.
- Advance-purchase-window pricing behavior (why T+1 is usually most expensive, dips around T+21-30, spikes near festivals) — helps you sanity-check your own scraped

data.

17. Team Role Suggestions (typical 6-member SIH team)

Role	Responsibility
Scraping Lead	Playwright/Scrapy scrapers, anti-bot/rate-limit strategy
Data Engineer	Cleaning pipeline, DB schema, data quality checks
Data Scientist / Statistician	Index methodology, weighting, backtesting, anomaly detection
Backend Developer	FastAPI, scheduler (Airflow/Celery), API docs
Frontend Developer	Dashboard (React + charting)
Presenter/PM	Documentation, pitch deck, demo choreography, Q&A prep

18. Suggested Repository Structure

```
apix/
├── scrapers/
│   ├── airlines/ (indigo.py, air_india.py, akasa.py, spicejet.py, air_india_express.py)
│   ├── otas/ (makemytrip.py, yatra.py, easemytrip.py, ...)
│   └── base_scraper.py
├── pipeline/
│   ├── cleaning.py
│   ├── outlier_detection.py
│   └── schema.py
├── index/
│   ├── weights.py (DGCA-based route weights)
│   ├── laspeyres.py / fisher.py
│   └── backtest.py
├── api/
│   └── main.py (FastAPI)
├── dashboard/
│   └── (React app)
├── tests/
├── docs/
│   ├── architecture.md
│   ├── methodology.md
│   └── ethics_compliance.md
└── README.md
```

19. Final Notes

- The single biggest credibility-builder for this specific PS is the **backtest correlation chart against DGCA data** — invest disproportionate effort here.

- The single biggest risk-reducer is an explicit, well-documented **ethics/compliance stance** on scraping — MoSPI is a government body and will care deeply about this.
- Don't over-promise "we scrape all 5 airlines + 6 OTAs flawlessly" if the demo can't back it up live — a smaller, rock-solid, well-explained pipeline beats a large, flaky one in front of judges.